School of Future Tech

Case Study Report

On

Traffic Signal Simulation System

By

KARTIK PRAVIN WAGH

150096725182

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study :

Traffic signals are used on roads to control vehicles and reduce accidents. A traffic signal changes between Red, Yellow, and Green lights to manage traffic flow.

This case study explains how to create a Traffic Signal Simulation System using C++. The program simulates how a real traffic signal works by changing signal colors automatically after a certain time interval. It helps students understand real-time system simulation and timing logic in programming.

2. Problem Statement / Case Background (Abstract) :

Traffic management is important in cities to control vehicle movement and avoid accidents. Traffic signals operate in a sequence where green allows vehicles to move, yellow warns drivers to slow down, and red stops vehicles.

The goal of this case study is to develop a C++ program that simulates a traffic signal system. The system will automatically change signal states after fixed time intervals and display the current signal on the screen.

This project demonstrates how programming concepts like loops, conditions, and time delay functions can be used to simulate real-world systems.

3. Problem Statement/Case Study Design :

The traffic signal system follows a simple design.

Signal States

The traffic signal has three main states:

- **Green** – Vehicles can move.
- **Yellow** – Warning signal to slow down.
- **Red** – Vehicles must stop.

System Flow

1. The program starts the traffic simulation.
2. It displays the **Green signal** first.
3. After a few seconds, it changes to **Yellow**.
4. Then it changes to **Red**.
5. The process repeats continuously until the user stops the program.

Menu Options

The system may include a simple menu:

- Start Signal
- Stop Signal
- Exit Program

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study :

Programming Language

- **C++**

Concepts Used

- **Loops** – To repeat the signal cycle continuously.
- **Conditional Statements** – To control signal switching.
- **Delay Function** – To simulate time intervals between signals.
- **Console Output** – To display the current signal state.

Simple Algorithm

1. Start the program.
2. Display menu options.

3. If user selects **Start Signal**:
    - Display **Green signal**.
    - Wait for a few seconds.
    - Display **Yellow signal**.
    - Wait for a few seconds.
    - Display **Red signal**.
    - Wait for a few seconds.
4. Repeat the cycle.
5. Stop when the user chooses **Exit**.

5. Problem Statement / Case Study Implementation Details and Snapshots

CODE :

C++ > signal.cpp > main()

```cpp
#include <iostream>
#include <thread>
#include <chrono>

using namespace std;

bool running = false;

void signalSystem()
{
    while (running)
    {
        cout << "Signal: GREEN (Go!)" << endl;
        this_thread::sleep_for(chrono::seconds(3));

        if (!running) break;

        cout << "Signal: YELLOW (Slow Down)" << endl;
        this_thread::sleep_for(chrono::seconds(2));

        if (!running) break;

        cout << "Signal: RED (Stop)" << endl;
        this_thread::sleep_for(chrono::seconds(3));
    }
}

int main()
{
    int choice;
    thread signalThread;

    cout << "===== Traffic Signal Simulation =====" << endl;

    while (true)
    {
        cout << "\nMenu" << endl;
        cout << "1. Start Signal" << endl;
        cout << "2. Stop Signal" << endl;
        cout << "3. Exit" << endl;
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1)
        {
            if (!running)
            {
                running = true;
                signalThread = thread(signalSystem);
                cout << "Signal Started...\n";
            }
            else
            {
                cout << "Signal already running!\n";
            }
```

```
        cin >> choice;

        if (choice == 1)
        {
            if (!running)
            {
                running = true;
                signalThread = thread(signalSystem);
                cout << "Signal Started...\n";
            }
            else
            {
                cout << "Signal already running!\n";
            }
        }
        else if (choice == 2)
        {
            running = false;

            if (signalThread.joinable())
                signalThread.join();

            cout << "Signal Stopped.\n";
        }
        else if (choice == 3)
        {
            running = false;

            if (signalThread.joinable())
                signalThread.join();

            cout << "Exiting program...\n";
            break;
        }
        else
        {
            cout << "Invalid choice!\n";
        }
    }

    return 0;
}
```

OUTPUT :

```
kartikwagh@Kartiks-MacBook-Air output % ./"signal"
===== Traffic Signal Simulation =====

Menu
1. Start Signal
2. Stop Signal
3. Exit
Enter choice: 1
Signal Started...

MenuSignal: GREEN (Go!)
1. Start Signal
2. Stop Signal

3. Exit
Enter choice: Signal: YELLOW (Slow Down)
Signal: RED (Stop)
Signal: GREEN (Go!)
Signal: YELLOW (Slow Down)
Signal: RED (Stop)
Signal: GREEN (Go!)
Signal: YELLOW (Slow Down)
Signal: RED (Stop)
Signal: GREEN (Go!)
Signal: YELLOW (Slow Down)
Signal: RED (Stop)
Signal: GREEN (Go!)
^C
```

6. Case Study Results and Conclusion :

Results :

The program successfully simulates a traffic signal system. It shows how signals change automatically with time delays. The system follows the correct order:

Green → Yellow → Red → Repeat

Conclusion :

This case study helped in understanding how real-world systems like traffic signals can be simulated using programming. It also improved knowledge of loops, control structures, and time-based operations in C++.

Such simulations can be extended further to create more complex traffic management systems.

7. Reference :

- C++ Programming Books and Tutorials
- ITM Skills University Course Material
- Online Programming Documentation
- www.w3schools.com (C++ Reference)